

Trust boundary for the exact certificate

Repository source: `certificate/TRUST_BOUNDARY.md`

Contents

Purpose	1
Two-part proof structure	1
Finite proof authority	1
Trusted computing base	2
Exact and non-exact operations	2
Exact theorem-bearing operations	2
Independent but non-authoritative operations	2
Immutable-object checks	2
Adversarial tests	3
What remains outside the exact machine boundary	3
Threat model	3
Recommended external reproduction	3

Purpose

This document states exactly what must be trusted for the finite certificate, what is checked independently, and what remains outside the machine-checked boundary.

The package represents the proposed theorem

$$\text{Vol}(K) \geq \frac{26220940089713\pi}{200000000000000} d^3$$

for every three-dimensional convex body K of constant width $d > 0$.

Two-part proof structure

The complete argument has two logically different parts.

1. **Human mathematical reduction.** The paper derives the constant-width identities, circumradius split, radial-propagation inequalities, contact-coordinate formulas, determinant transfer, coordinate-width lemma, cone-cap construction, and the final branch assembly.
2. **Finite exact verification.** The programs in `certificate/` check the resulting finite rational inequalities, immutable arrays, hulls, cap allocations, disjointness inequalities, and exact theorem coefficient.

The finite verifier does not by itself prove that every mathematical reduction in the paper is valid. Conversely, the prose argument does not replace replay of the large finite certificate.

Finite proof authority

The finite proof authority consists of all of the following together:

- `certificates/universal_contact_bound_certificate.json`;
- `certificates/reference_handoff_certificate.json`;
- `certificates/reference_near_jung_fan.npz`;
- `certificates/universal_contact_fan.npz`;
- `certificates/refined_lower_cell_1.npz` through `refined_lower_cell_4.npz`;

- the exact TSV and CSV renderings under `certificates/`;
- `certify_universal_bound.py`;
- `exact_geometry.py`;
- the SHA-256 manifests; and
- byte-identical reconstruction checks performed by the verifier.

No single decimal transcript is sufficient proof authority. The exact certificate must be accepted by the exact program from checksum-verified objects.

Trusted computing base

The exact path assumes correct behavior of:

- the operating system's ordinary file-reading operations;
- SHA-256 as implemented by Python `hashlib` and the shell checksum utility;
- the CPython interpreter;
- arbitrary-precision Python integers;
- `Fractions.Fraction` arithmetic and comparisons;
- Python's standard JSON, text, and path handling;
- NumPy 2.3.5 for parsing `allow_pickle=False` NPZ files and presenting their fixed-width integer and string arrays;
- conversion of loaded `int64` values to Python integers before theorem-bearing arithmetic; and
- the exact source files committed in the repository.

NumPy is not used to justify a theorem-bearing floating-point comparison in the exact path. It is nevertheless part of the trusted parser boundary because the proof arrays are stored in NPZ containers.

Exact and non-exact operations

Exact theorem-bearing operations

The verifier uses Python integers and `Fraction` objects for:

- lower-row propagation inequalities;
- fan radial and multiplier checks;
- S-procedure margins;
- determinant and coordinate-width estimates;
- section-hull areas;
- cap-depth and cap-volume bounds;
- cross-axis disjointness;
- branch comparison; and
- the theorem coefficient.

Exact decimal strings are generated only after the rational comparison has been completed. Decimal formatting is not the logical basis of the theorem.

Independent but non-authoritative operations

The following improve confidence but are not finite proof authority:

- `audit_universal_bound.py`, which uses binary64 NumPy arithmetic;
- `audit_universal_bound.R`, which uses base-R numerical arithmetic;
- the rendered paper and documentation PDFs;
- visual page inspection;
- timing measurements;
- GitHub Actions status by itself; and
- any AI-generated explanation.

Immutable-object checks

The exact verifier checks declared SHA-256 fingerprints before semantic use of key predecessor and fan objects. Repository manifests protect the remaining tracked inputs and transcripts. Derived JSON, CSV, and TSV artifacts

are rebuilt and compared byte for byte.

The checksum layer detects accidental or unauthorized byte changes. It does not show that the original object was mathematically constructed correctly; that is why the semantic verifier and independent audits are also required.

Adversarial tests

`test_adversarial_mutations.py` changes temporary copies of seven proof-critical objects and requires rejection of:

1. a lower-row endpoint mutation;
2. a fan radial-numerator mutation;
3. an S-procedure multiplier mutation;
4. a section-hull vertex mutation;
5. a cap-allocation bracket mutation;
6. a disjointness-threshold mutation; and
7. a theorem-coefficient digit mutation.

These tests show that the named gates are active. They do not prove the absence of every possible verifier defect.

What remains outside the exact machine boundary

The following require mathematical or release review beyond the finite replay:

- correctness of the paper’s geometric reductions;
- completeness of the reduction from arbitrary convex bodies to the certified branch cases;
- correctness of cited classical theorems;
- novelty and priority;
- the interpretation of the certificate fields in the manuscript;
- equality, rigidity, sharpness, and minimizer questions;
- Meissner extremality;
- publication metadata, licensing interpretation, and release administration; and
- independent peer or expert review.

Threat model

The package is designed primarily to detect:

- accidental file corruption;
- stale generated artifacts;
- paper/certificate drift;
- silent changes to exact arrays;
- common implementation errors exposed by an independent language path; and
- verifier paths that fail to reject targeted corruptions.

It is not designed to defend against a compromised operating system, malicious Python or NumPy distribution, broken cryptographic primitives, or coordinated changes to both verifier and expected outputs. Reviewers concerned with those risks should replay in independently sourced environments and inspect the mathematical reductions and code.

Recommended external reproduction

A strong external reproduction should:

1. obtain the immutable tagged release independently;
2. verify release-asset and repository SHA-256 manifests;
3. inspect the paper’s geometric reduction;
4. run the exact verifier in a fresh Python environment;
5. run the base-R audit independently;
6. examine at least one proof-object class directly;
7. rerun the mutation suite;
8. compare generated and committed exact artifacts; and
9. report the exact tag, commit, operating system, Python, NumPy, and R versions.

A successful reproduction should still be described as reproduction of the package unless the reviewer has also checked the mathematics independently.